

Job Card Management System - Backend Technical Documentation

Developer: Solo Project
Technology Stack: Node.js, Express.js, PostgreSQL, JWT
Status: Development (with mock data)
Presentation Duration: 1-2 hours
Audience: Senior Software Engineers (Technical Review Panel)

Table of Contents

- 1. [System Overview](#)
 - 2. [Architecture & Design Patterns](#)
 - 3. [Database Design](#)
 - 4. [Authentication & Authorization](#)
 - 5. [API Design & Routing](#)
 - 6. [Business Logic Layer](#)
 - 7. [Data Validation & Error Handling](#)
 - 8. [Security Implementation](#)
 - 9. [Key Technical Decisions](#)
 - 10. [Code Walkthrough](#)
 - 11. [Testing & Deployment](#)
 - 12. [Future Improvements](#)
-

1. System Overview

1.1 Problem Statement

Copy Cat Group sends technicians to customer sites for photocopier installations, maintenance, and repairs. The existing paper-based job card system suffers from:

- **Lost or delayed job cards** (no central tracking)
- **Lack of real-time visibility** (supervisors can't monitor progress)
- **Poor accountability** (no audit trail)
- **Data fragmentation** (reports stored in multiple locations)

1.2 Solution

A digital job card management system with:

- **Role-based dashboards** (Technician vs Supervisor views)
- **Real-time job tracking** (pending → in progress → completed)
- **Audit trail** (status history logging)
- **RESTful API** (stateless, scalable architecture)

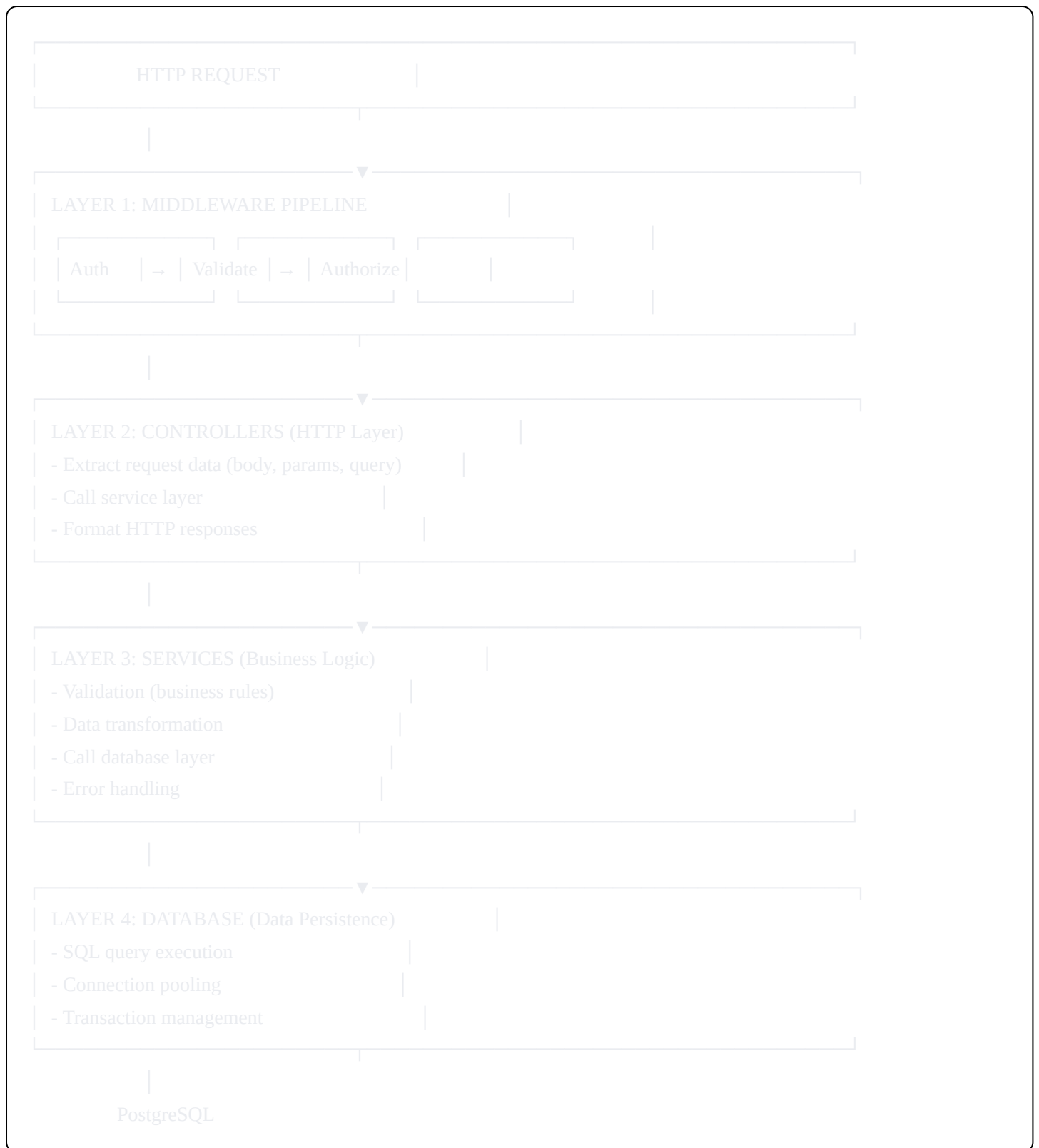
1.3 Tech Stack Justification

Technology	Reason for Selection
Node.js	Non-blocking I/O ideal for API servers; JavaScript across stack
Express.js	Minimal, unopinionated; excellent middleware ecosystem
PostgreSQL	ACID compliance; complex relationships; data integrity constraints
JWT	Stateless authentication; scalable across multiple servers
Bcrypt	Industry-standard password hashing; configurable salt rounds
Joi	Schema-based validation; clear error messages

2. Architecture & Design Patterns

2.1 Layered Architecture

The backend follows a **4-layer architecture** to separate concerns:



Why This Architecture?

1. **Separation of Concerns:** Each layer has a single responsibility
2. **Testability:** Layers can be unit tested independently
3. **Maintainability:** Changes in one layer don't cascade
4. **Scalability:** Layers can be horizontally scaled separately

2.2 Request Flow Example

Let's trace a request to **start a job card**:



2.3 Design Patterns Used

2.3.1 Repository Pattern (Implicit)

While not explicitly named, the **service layer** acts as a repository:

```
javascript

// Services abstract database operations
jobCardService.getById(id)    // Hides SQL implementation
jobCardService.create(data)   // Centralizes validation
jobCardService.update(id, data) // Single source of truth
```

Benefits:

- Database logic stays out of controllers
- Easy to mock for testing
- Can swap databases without changing controllers

2.3.2 Middleware Chain Pattern

Express middleware forms a **Chain of Responsibility**:

javascript

```

router.patch('/:id',
  authenticate,      // Step 1: Who are you?
  verifyJobCardOwnership, // Step 2: Do you own this?
  validateRequest(schema), // Step 3: Is data valid?
  updateJobCard      // Step 4: Process request
);

```

Each middleware decides: **process and pass** or **reject and respond**.

2.3.3 Factory Pattern (Error Handling)

Custom error creation:

```

javascript

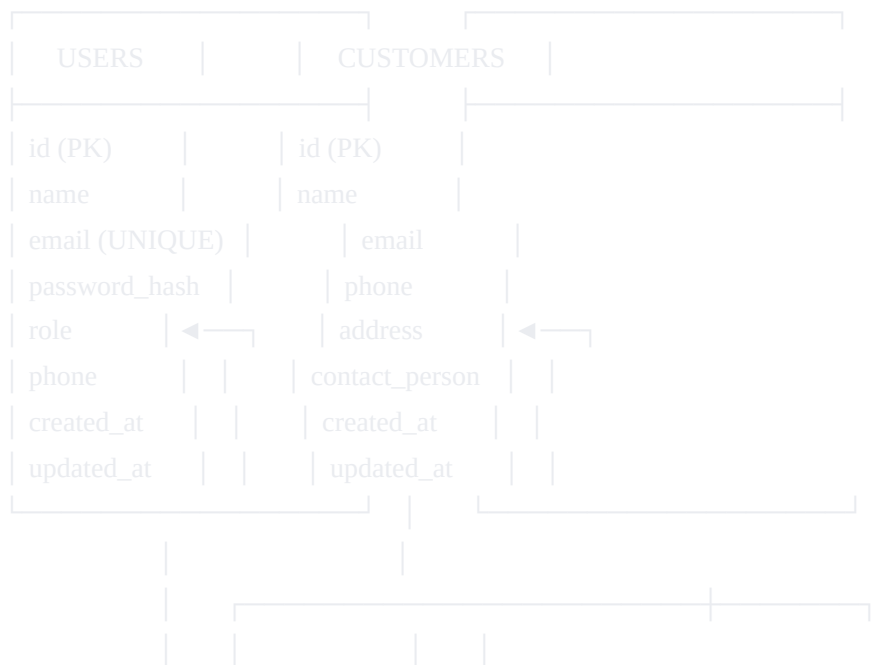
// services/jobCard.service.js
const error = new Error('Job card not found');
error.statusCode = 404; // Factory adds properties
throw error;

// Caught by error handler middleware
res.status(error.statusCode || 500).json({
  success: false,
  error: error.message
});

```

3. Database Design

3.1 Entity Relationship Diagram





3.2 Table Schemas

3.2.1 Users Table

sql

```

CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL CHECK (role IN ('technician', 'supervisor')),
  phone VARCHAR(20),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for performance
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);

```

Key Design Decisions:

1. **Email as unique identifier:** Natural key for authentication
2. **Role constraint:** Only 2 roles (simplifies RBAC)
3. **No soft deletes:** RESTRICT on foreign keys prevents orphaned records
4. **Timestamps:** Audit trail for account creation/updates

3.2.2 Customers Table

```

sql

CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  address TEXT NOT NULL,
  phone VARCHAR(20) NOT NULL,
  email VARCHAR(255),
  contact_person VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_customers_name ON customers(name);

```

Key Design Decisions:

1. **Separate from users:** Customers aren't system users (no login)
2. **Nullable email:** Some customers may only have phone contact
3. **contact_person optional:** Useful for corporate clients

3.2.3 Job Cards Table (Core Entity)

sql

```
CREATE TABLE job_cards (  
  id SERIAL PRIMARY KEY,  
  customer_id INTEGER NOT NULL REFERENCES customers(id) ON DELETE RESTRICT,  
  technician_id INTEGER NOT NULL REFERENCES users(id) ON DELETE RESTRICT,  
  title VARCHAR(200) NOT NULL,  
  description TEXT,  
  status VARCHAR(20) NOT NULL DEFAULT 'pending'  
    CHECK (status IN ('pending', 'in_progress', 'completed')),  
  priority VARCHAR(20) NOT NULL DEFAULT 'medium'  
    CHECK (priority IN ('low', 'medium', 'high', 'urgent')),  
  scheduled_date TIMESTAMP NOT NULL,  
  estimated_duration INTEGER, -- minutes  
  actual_start_time TIMESTAMP,  
  actual_end_time TIMESTAMP,  
  work_performed TEXT,  
  notes TEXT,  
  customer_signature TEXT, -- base64 encoded (future feature)  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  completed_at TIMESTAMP,  
  
  -- Constraint: Only technicians can be assigned  
  CONSTRAINT valid_technician_role CHECK (  
    (SELECT role FROM users WHERE id = technician_id) = 'technician'  
  ),  
  
  -- Constraint: End time must be after start time  
  CONSTRAINT valid_times CHECK (  
    actual_end_time IS NULL OR actual_start_time IS NULL  
    OR actual_end_time >= actual_start_time  
  )  
);  
  
-- Performance indexes  
CREATE INDEX idx_job_cards_status ON job_cards(status);  
CREATE INDEX idx_job_cards_technician ON job_cards(technician_id);  
CREATE INDEX idx_job_cards_customer ON job_cards(customer_id);  
CREATE INDEX idx_job_cards_scheduled_date ON job_cards(scheduled_date);
```

Key Design Decisions:

1. **Status enum:** Only 3 states (simple state machine)
2. **RESTRICT on deletes:** Cannot delete customer/tech with active jobs

3. **Nullable actual times:** Only populated when job starts/ends
4. **CHECK constraints:** Database-level validation (defense in depth)
5. **customer_signature:** Prepared for future PDF generation

3.2.4 Job Status History Table

sql

```
CREATE TABLE job_status_history (  
  id SERIAL PRIMARY KEY,  
  job_card_id INTEGER NOT NULL REFERENCES job_cards(id) ON DELETE CASCADE,  
  status VARCHAR(20) NOT NULL  
    CHECK (status IN ('pending', 'in_progress', 'completed')),  
  changed_by_user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE RESTRICT,  
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  notes TEXT  
);  
  
CREATE INDEX idx_status_history_job_card ON job_status_history(job_card_id);  
CREATE INDEX idx_status_history_changed_at ON job_status_history(changed_at);
```

Key Design Decisions:

1. **CASCADE on job card delete:** History is meaningless without parent job
2. **changed_by_user_id:** Audit trail (who changed it?)
3. **Immutable:** No UPDATE/DELETE operations (append-only log)

3.3 Database Triggers (Automation)

3.3.1 Auto-Update Timestamps

sql

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ language 'plpgsql';
```

```
CREATE TRIGGER update_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
```

```
CREATE TRIGGER update_customers_updated_at
BEFORE UPDATE ON customers
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
```

```
CREATE TRIGGER update_job_cards_updated_at
BEFORE UPDATE ON job_cards
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
```

Why Triggers for Timestamps?

- **Automatic:** Developers can't forget to set `updated_at`
- **Consistent:** All updates use same timestamp logic
- **Database-level:** Works even with direct SQL updates

3.3.2 Auto-Log Status Changes

```
sql
```

```

CREATE OR REPLACE FUNCTION log_job_status_change()
RETURNS TRIGGER AS $$
BEGIN
  IF (TG_OP = 'UPDATE' AND OLD.status != NEW.status) THEN
    INSERT INTO job_status_history (job_card_id, status, changed_by_user_id)
    VALUES (NEW.id, NEW.status, NEW.technician_id);
  END IF;
  RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER log_status_change
AFTER UPDATE ON job_cards
FOR EACH ROW
EXECUTE FUNCTION log_job_status_change();

```

Critical Insight:

This trigger creates an **audit trail automatically**. Every status change is logged WITHOUT requiring application code. This is a **defense-in-depth** strategy:

- If a bug bypasses logging in the application → trigger catches it
- Direct database updates → still logged
- Provides forensic data for debugging

3.4 Connection Pooling

```

javascript

// config/database.js
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,           // Max 20 concurrent connections
  idleTimeoutMillis: 30000, // Close idle connections after 30s
  connectionTimeoutMillis: 2000, // Fail fast if can't connect
});

pool.on('error', (err) => {
  console.error('Unexpected database error:', err);
  process.exit(-1); // Crash to trigger restart (fail-fast)
});

module.exports = pool;

```

Why Connection Pooling?

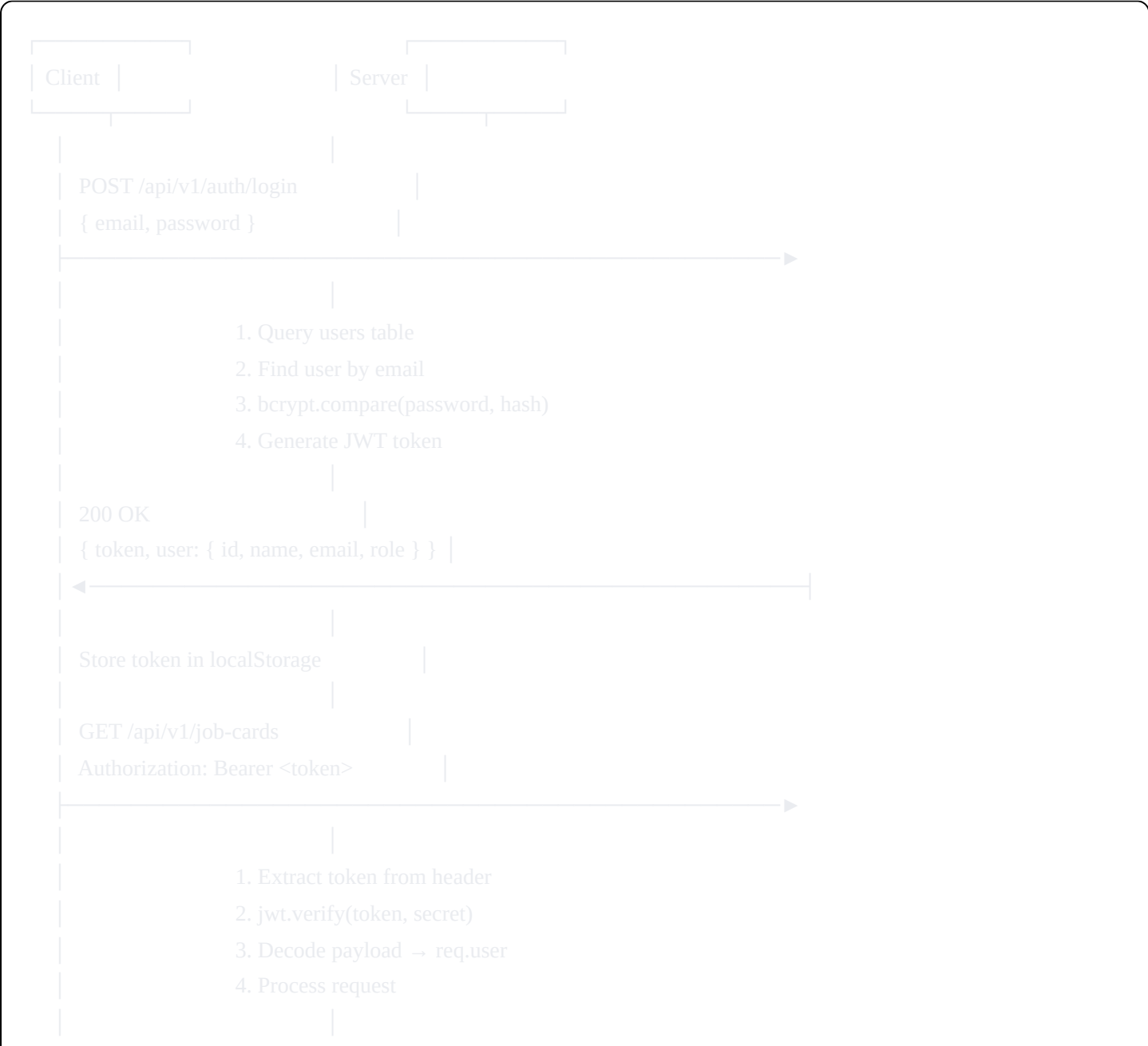
- 1. **Performance:** Reuse connections (TCP handshake is expensive)
- 2. **Resource management:** Limit concurrent connections to database
- 3. **Reliability:** Auto-reconnect on connection loss

Configuration Rationale:

- **max: 20:** Supports 20 concurrent requests (adjust based on server resources)
 - **idleTimeoutMillis: 30000:** Free up connections during low traffic
 - **connectionTimeoutMillis: 2000:** Fail fast (don't hang on database issues)
-

4. Authentication & Authorization

4.1 Authentication Flow



200 OK

{ data: [...] }



4.2 Password Hashing

javascript

// utils/authHelpers.js

```
const bcrypt = require('bcrypt');
```

```
const hashPassword = async (password) => {  
  const saltRounds = parseInt(process.env.BCRYPT_ROUNDS) || 10;  
  return await bcrypt.hash(password, saltRounds);  
};
```

```
const comparePassword = async (password, hash) => {  
  return await bcrypt.compare(password, hash);  
};
```

Why Bcrypt?

1. **Adaptive:** Salt rounds can increase as hardware improves
2. **Salt included:** Each hash is unique (prevents rainbow tables)
3. **Slow by design:** Mitigates brute-force attacks

Salt Rounds Explanation:

- `saltRounds = 10` → $2^{10} = 1,024$ iterations
- Higher = slower (more secure, but impacts performance)
- 10 is industry standard balance

4.3 JWT Token Strategy

javascript

```
// utils/authHelpers.js
const jwt = require('jsonwebtoken');

const generateToken = (payload) => {
  const secret = process.env.JWT_SECRET;
  const expiresIn = process.env.JWT_EXPIRY || '24h';

  return jwt.sign(payload, secret, { expiresIn });
};

// Payload structure
{
  userId: 123,
  email: "tech@copycat.com",
  role: "technician",
  iat: 1706184000, // Issued at
  exp: 1706270400 // Expires (24h later)
}
```

Why JWT?

1. **Stateless:** No server-side session storage required
2. **Scalable:** Works across multiple servers (no shared session state)
3. **Self-contained:** Payload contains user info (no database lookup)

Security Considerations:

- **Secret in environment variable:** Never hardcode
- **24h expiry:** Balance between UX and security
- **No sensitive data in payload:** JWTs are base64 (not encrypted)

4.4 Authentication Middleware

```
javascript
```

```
// middleware/authenticate.js

const { verifyToken } = require('../utils/authHelpers');

const authenticate = async (req, res, next) => {
  try {
    // Extract token from header
    const authHeader = req.headers.authorization;

    if (!authHeader) {
      return res.status(401).json({
        success: false,
        error: 'No authorization token provided'
      });
    }

    // Check "Bearer <token>" format
    if (!authHeader.startsWith('Bearer ')) {
      return res.status(401).json({
        success: false,
        error: 'Invalid authorization format'
      });
    }

    const token = authHeader.substring(7); // Remove "Bearer "

    // Verify and decode token
    const decoded = verifyToken(token);

    // Attach user info to request object
    req.user = {
      userId: decoded.userId,
      email: decoded.email,
      role: decoded.role
    };

    next(); // Continue to next middleware

  } catch (error) {
    // Handle specific JWT errors
    if (error.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        error: 'Token expired',
        message: 'Please login again'
      });
    }
  }
}
```

```
if (error.name === 'JsonWebTokenError') {  
  return res.status(401).json({  
    success: false,  
    error: 'Invalid token'  
  });  
}  
  
return res.status(401).json({  
  success: false,  
  error: 'Authentication failed'  
});  
}  
};
```

Critical Points:

1. **Early return on failure:** Don't call `next()` on auth errors
2. **Set req.user:** Makes user info available to downstream middleware/controllers
3. **Specific error messages:** Help debugging (but don't leak security info)

4.5 Authorization Middleware

javascript

```
// middleware/authorize.js
const authorize = (allowedRoles) => {
  return (req, res, next) => {
    // Check if user was authenticated
    if (!req.user) {
      return res.status(401).json({
        success: false,
        error: 'Authentication required'
      });
    }

    // Check if user's role is allowed
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        error: 'Insufficient permissions',
        message: `Requires role: ${allowedRoles.join(' or ')}`
      });
    }

    next(); // User has required role
  };
};

// Usage example
router.post('/job-cards',
  authenticate,           // Step 1: Verify token
  authorize(['supervisor']), // Step 2: Check role
  createJobCard           // Step 3: Process request
);
```

401 vs 403:

- **401 Unauthorized:** No valid credentials (missing/invalid token)
- **403 Forbidden:** Valid credentials, but insufficient permissions

4.6 Job Card Ownership Verification

javascript

```

// middleware/jobCardAuth.js
const verifyJobCardOwnership = async (req, res, next) => {
  try {
    const jobCardId = parseInt(req.params.id);
    const { userId, role } = req.user;

    // Supervisors bypass ownership check
    if (role === 'supervisor') {
      return next();
    }

    // Technicians: verify they own this job
    const result = await pool.query(
      'SELECT technician_id FROM job_cards WHERE id = $1',
      [jobCardId]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({
        success: false,
        error: 'Job card not found'
      });
    }

    const jobTechnicianId = result.rows[0].technician_id;

    if (jobTechnicianId !== userId) {
      return res.status(403).json({
        success: false,
        error: 'You can only access your own assigned job cards'
      });
    }

    next(); // Ownership verified

  } catch (error) {
    console.error('Ownership verification error:', error);
    res.status(500).json({
      success: false,
      error: 'Failed to verify job card ownership'
    });
  }
};

```

Why Separate Ownership Middleware?

1. **Reusable:** Apply to GET, PATCH, POST endpoints
2. **Declarative:** Route definition shows authorization logic
3. **Testable:** Can unit test ownership logic independently

4.7 Role-Based Access Control Matrix

Endpoint	Technician	Supervisor
POST /auth/login	✓	✓
POST /auth/register	✓	✓
GET /job-cards	✓ (own jobs only)	✓ (all jobs)
GET /job-cards/:id	✓ (if assigned)	✓
POST /job-cards	✗	✓
PATCH /job-cards/:id	✓ (own jobs, work details only)	✓
POST /job-cards/:id/complete	✓ (if assigned)	✓
DELETE /job-cards/:id	✗	✓ (pending only)
GET /customers	✓ (read-only)	✓
POST /customers	✗	✓
GET /users	✗	✓
DELETE /users/:id	✗	✓

5. API Design & Routing

5.1 RESTful Principles

The API follows REST conventions:

1. **Resource-based URLs:** `/job-cards` not `/getJobCards`
2. **HTTP verbs:** GET (read), POST (create), PATCH (update), DELETE (remove)
3. **Stateless:** Each request contains all needed information
4. **JSON responses:** Consistent format across endpoints

5.2 API Structure

```
/api/v1
├── /auth
│   ├── POST /register    # Create account
│   ├── POST /login      # Get JWT token
│   ├── GET  /me         # Get current user
│   └── POST /logout      # Clear session (client-side)
├── /users
│   ├── GET  /users       # List all (supervisor only)
│   ├── GET  /users/stats # User statistics
│   ├── GET  /users/:id   # Get single user
│   ├── PATCH /users/:id  # Update user
│   └── DELETE /users/:id  # Delete user (supervisor only)
├── /customers
│   ├── GET  /customers   # List all
│   ├── GET  /customers/stats # Customer statistics
│   ├── GET  /customers/:id # Get single customer
│   ├── POST /customers   # Create (supervisor only)
│   ├── PATCH /customers/:id # Update (supervisor only)
│   └── DELETE /customers/:id # Delete (supervisor only)
└── /job-cards
    ├── GET  /job-cards    # List with filters
    ├── GET  /job-cards/stats # Statistics (supervisor only)
    ├── GET  /job-cards/:id # Get single job
    ├── POST /job-cards     # Create (supervisor only)
    ├── PATCH /job-cards/:id # Update
    ├── POST /job-cards/:id/complete # Complete job
    └── DELETE /job-cards/:id # Delete (supervisor, pending only)
```

5.3 Route Implementation Example

```
javascript
```

```
// routes/jobCard.routes.js

const express = require('express');
const router = express.Router();
const jobCardController = require('../controllers/jobCard.controller');
const authenticate = require('../middleware/authenticate');
const authorize = require('../middleware/authorize');
const validateRequest = require('../middleware/validateRequest');
const { verifyJobCardOwnership, filterJobCardsByRole } = require('../middleware/jobCardAuth');
const {
  createJobCardSchema,
  updateJobCardSchema,
  completeJobCardSchema,
  getJobCardsQuerySchema
} = require('../validators/jobCard.validator');

// Statistics (supervisor only)
router.get('/stats',
  authenticate,
  authorize(['supervisor']),
  jobCardController.getJobCardStatistics
);

// List all job cards
router.get('/',
  authenticate,
  filterJobCardsByRole, // Auto-filter for technicians
  validateRequest(getJobCardsQuerySchema),
  jobCardController.getAllJobCards
);

// Get single job card
router.get('/:id',
  authenticate,
  verifyJobCardOwnership, // Check ownership
  jobCardController.getJobCardById
);

// Create new job card
router.post('/',
  authenticate,
  authorize(['supervisor']),
  validateRequest(createJobCardSchema),
  jobCardController.createJobCard
);

// Update job card
```

```
router.patch('/:id',
  authenticate,
  verifyJobCardOwnership,
  validateRequest(updateJobCardSchema),
  jobCardController.updateJobCard
);

// Complete job card
router.post('/:id/complete',
  authenticate,
  verifyJobCardOwnership,
  validateRequest(completeJobCardSchema),
  jobCardController.completeJobCard
);

// Delete job card
router.delete('/:id',
  authenticate,
  authorize(['supervisor']),
  jobCardController.deleteJobCard
);

module.exports = router;
```

Route Design Decisions:

1. **Stats before :id:** Specific routes before parameterized routes (avoid conflicts)
2. **Middleware order matters:** authenticate → authorize → validate → controller
3. **Separate complete endpoint:** `/complete` is more semantic than PATCH with status
4. **Query validation:** GET endpoints also validate query parameters

5.4 Query Parameter Filtering

javascript

```
// GET /api/v1/job-cards?status=pending&technician_id=5&page=2&limit=20
```

```
// Service layer handles filtering
```

```
const getAllJobCards = async (filters = {}) => {  
  const {  
    status,          // 'pending', 'in_progress', 'completed', or array  
    technician_id,   // Filter by technician  
    customer_id,     // Filter by customer  
    start_date,      // Date range start  
    end_date,        // Date range end  
    search,          // Search in title/description  
    page = 1,        // Pagination  
    limit = 20  
  } = filters;  
  
  let query = `SELECT ... FROM job_cards ...`;  
  const conditions = [];  
  const params = [];  
  let paramCounter = 1;  
  
  // Build WHERE clause dynamically  
  if (status) {  
    if (Array.isArray(status)) {  
      // Multiple statuses: status IN ('pending', 'in_progress')  
      const placeholders = status.map((_, i) => `${paramCounter + i}`).join(', ');  
      conditions.push(`jc.status IN (${placeholders})`);  
      params.push(...status);  
      paramCounter += status.length;  
    } else {  
      // Single status  
      conditions.push(`jc.status = ${paramCounter}`);  
      params.push(status);  
      paramCounter++;  
    }  
  }  
  
  // ... more filters  
  
  if (conditions.length > 0) {  
    query += ' WHERE ' + conditions.join(' AND ');  
  }  
  
  // Add pagination  
  const offset = (page - 1) * limit;  
  query += ` LIMIT ${paramCounter} OFFSET ${paramCounter + 1}`;  
  params.push(limit, offset);  
}
```

```
const result = await pool.query(query, params);  
// ...  
};
```

Why Dynamic Query Building?

1. **Flexible filtering:** Clients can combine filters as needed
2. **Performance:** Only applies necessary conditions
3. **SQL injection safe:** Uses parameterized queries

5.5 Response Format Standards

javascript

```
// Success response
{
  "success": true,
  "message": "Job card created successfully", // Optional
  "data": {
    "jobCard": {
      "id": 42,
      "title": "HVAC Maintenance",
      // ... more fields
    }
  }
}
```

```
// Error response
{
  "success": false,
  "error": "Validation failed",
  "details": [
    {
      "field": "scheduled_date",
      "message": "scheduled_date is required"
    }
  ]
}
```

```
// Paginated response
{
  "success": true,
  "data": [...],
  "pagination": {
    "currentPage": 2,
    "totalPages": 5,
    "totalCount": 87,
    "limit": 20
  }
}
```

Consistency Benefits:

- Frontend can rely on `success` boolean
 - `data` always contains the payload
 - `pagination` present on list endpoints
 - `details` provides field-level errors
-

6. Business Logic Layer

6.1 Service Layer Responsibilities

The **service layer** implements all business logic:

1. **Validation:** Business rules (beyond schema validation)
2. **State management:** Job status transitions
3. **Database operations:** CRUD with complex logic
4. **Error handling:** Throw errors with appropriate status codes

6.2 Job Card Service - Core Logic

6.2.1 Create Job Card

```
javascript
```

```
// services/jobCard.service.js

const createJobCard = async (jobCardData) => {
  const {
    customer_id,
    technician_id,
    title,
    description,
    priority = 'medium',
    scheduled_date,
    estimated_duration,
    notes
  } = jobCardData;

  // VALIDATION 1: Verify customer exists
  const customerCheck = await pool.query(
    'SELECT id FROM customers WHERE id = $1',
    [customer_id]
  );

  if (customerCheck.rows.length === 0) {
    const error = new Error(`Customer with ID ${customer_id} does not exist`);
    error.statusCode = 400;
    throw error;
  }

  // VALIDATION 2: Verify technician exists AND has correct role
  const technicianCheck = await pool.query(
    'SELECT id, role FROM users WHERE id = $1',
    [technician_id]
  );

  if (technicianCheck.rows.length === 0) {
    const error = new Error(`User with ID ${technician_id} does not exist`);
    error.statusCode = 400;
    throw error;
  }

  if (technicianCheck.rows[0].role !== 'technician') {
    const error = new Error(
      `User with ID ${technician_id} is not a technician (role: ${technicianCheck.rows[0].role})`
    );
    error.statusCode = 400;
    throw error;
  }

  // CREATE JOB CARD
```

```

const query = `
  INSERT INTO job_cards (
    customer_id, technician_id, title, description,
    priority, scheduled_date, estimated_duration, notes
  ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
  RETURNING id, customer_id, technician_id, title, status, priority, created_at
`;

const values = [
  customer_id, technician_id, title, description || null,
  priority, scheduled_date, estimated_duration || null, notes || null
];

const result = await pool.query(query, values);

// FETCH COMPLETE JOB CARD WITH JOINS (customer + technician details)
return await getJobCardById(result.rows[0].id);
};

```

Business Rules Enforced:

1. **Foreign key validation:** Customer and technician must exist
2. **Role validation:** Only users with role='technician' can be assigned
3. **Default priority:** Ensures every job has a priority
4. **Return complete object:** Includes related entities (better UX)

6.2.2 Update Job Card - Status Transition Logic

javascript

```
const updateJobCard = async (jobCardId, updateData) => {
  // STEP 1: Fetch current job state
  const currentJobResult = await pool.query(
    'SELECT status, actual_start_time FROM job_cards WHERE id = $1',
    [jobCardId]
  );

  if (currentJobResult.rows.length === 0) {
    const error = new Error('Job card not found');
    error.statusCode = 404;
    throw error;
  }

  const currentJob = currentJobResult.rows[0];
  const currentStatus = currentJob.status;

  // VALIDATION 1: Prevent modification of completed jobs
  if (currentStatus === 'completed') {
    const error = new Error(
      'Cannot modify completed job cards. Completed jobs are immutable for data integrity.'
    );
    error.statusCode = 403; // Forbidden
    throw error;
  }

  // VALIDATION 2: Status transition logic
  if (updateData.status && updateData.status !== currentStatus) {
    const invalidTransitions = {
      'in_progress': ['pending'], // Can't go back to pending
      'completed': [] // Can't change from completed
    };

    if (invalidTransitions[currentStatus]?.includes(updateData.status)) {
      const error = new Error(
        `Invalid status transition: Cannot change from '${currentStatus}' to '${updateData.status}'`
      );
      error.statusCode = 400;
      throw error;
    }
  }

  // Actual start time required when starting a job
  if (updateData.status === 'in_progress' && !updateData.actual_start_time) {
    const error = new Error(
      'actual_start_time is required when changing status to in_progress'
    );
    error.statusCode = 400;
  }
}
```

```

        throw error;
    }
}

// VALIDATION 3: Time logic
if (updateData.actual_start_time && updateData.actual_end_time) {
    const startTime = new Date(updateData.actual_start_time);
    const endTime = new Date(updateData.actual_end_time);

    if (endTime < startTime) {
        const error = new Error('actual_end_time cannot be before actual_start_time');
        error.statusCode = 400;
        throw error;
    }
}

// BUILD UPDATE QUERY DYNAMICALLY
const allowedFields = [
    'title', 'description', 'status', 'priority',
    'scheduled_date', 'estimated_duration',
    'actual_start_time', 'actual_end_time',
    'work_performed', 'notes'
];

const updates = [];
const values = [];
let paramCounter = 1;

for (const field of allowedFields) {
    if (updateData[field] !== undefined) {
        updates.push(`${field} = ${updateData[field]}`);
        values.push(updateData[field]);
        paramCounter++;
    }
}

if (updates.length === 0) {
    // No fields to update, return current job
    return await getJobCardById(jobCardId);
}

values.push(jobCardId);

const query = `
    UPDATE job_cards
    SET ${updates.join(', ')}
    WHERE id = ${updateData.id}`;

```

```

    RETURNING id
  ;

  await pool.query(query, values);

  // Database trigger handles status history logging automatically
  return await getJobCardById(jobCardId);
};

```

State Machine Diagram:



Invalid Transitions:

- in_progress** → **pending** ❌ (can't un-start a job)
- completed** → ***** ❌ (completed jobs are read-only)

Why Immutable Completed Jobs?

- Data integrity:** Prevents tampering with historical records
- Audit compliance:** Completed work is a legal record
- Reporting accuracy:** Statistics won't change retroactively

6.2.3 Complete Job Card

javascript

```
const completeJobCard = async (jobCardId, completionData) => {
  // Fetch current job status
  const currentJobResult = await pool.query(
    'SELECT status, actual_start_time FROM job_cards WHERE id = $1',
    [jobCardId]
  );

  if (currentJobResult.rows.length === 0) {
    const error = new Error('Job card not found');
    error.statusCode = 404;
    throw error;
  }

  const currentJob = currentJobResult.rows[0];

  // VALIDATION 1: Check if already completed
  if (currentJob.status === 'completed') {
    const error = new Error('Job card is already completed');
    error.statusCode = 400;
    throw error;
  }

  // VALIDATION 2: Check if job was started first
  if (currentJob.status === 'pending') {
    const error = new Error(
      'Job must be started before it can be completed. Please start the job first.'
    );
    error.statusCode = 400;
    throw error;
  }

  // VALIDATION 3: Required fields for completion
  if (!completionData.work_performed) {
    const error = new Error('work_performed is required to complete a job');
    error.statusCode = 400;
    throw error;
  }

  if (completionData.work_performed.length < 10) {
    const error = new Error('work_performed must be at least 10 characters long');
    error.statusCode = 400;
    throw error;
  }

  // VALIDATION 4: Time logic
  const startTime = completionData.actual_start_time
```

```
? new Date(completionData.actual_start_time)
: currentJob.actual_start_time
? new Date(currentJob.actual_start_time)
: null;

const endTime = completionData.actual_end_time
? new Date(completionData.actual_end_time)
: new Date(); // Default to current time

if (!startTime) {
  const error = new Error(
    'Job must have actual_start_time before completion'
  );
  error.statusCode = 400;
  throw error;
}

if (endTime < startTime) {
  const error = new Error('actual_end_time cannot be before actual_start_time');
  error.statusCode = 400;
  throw error;
}

// UPDATE JOB CARD TO COMPLETED STATUS
const query = `
  UPDATE job_cards
  SET
    status = 'completed',
    actual_start_time = COALESCE($1, actual_start_time),
    actual_end_time = $2,
    work_performed = $3,
    customer_signature = $4,
    notes = COALESCE($5, notes),
    completed_at = CURRENT_TIMESTAMP
  WHERE id = $6
  RETURNING id
`;

const values = [
  completionData.actual_start_time || null,
  endTime,
  completionData.work_performed,
  completionData.customer_signature || null,
  completionData.notes || null,
  jobCardId
];
```

```
await pool.query(query, values);

// Database trigger logs completion to job_status_history
return await getJobCardById(jobCardId);
};
```

Completion Requirements:

1. **Job must be in_progress:** Can't complete a pending job
2. **work_performed required:** Must document what was done (min 10 chars)
3. **actual_start_time required:** Either from DB or provided
4. **actual_end_time:** Defaults to now if not provided
5. **customer_signature optional:** Future feature for digital sign-off

6.2.4 Delete Job Card

javascript

```

const deleteJobCard = async (jobCardId) => {
  // Check if job card exists and get its status
  const jobCheck = await pool.query(
    'SELECT id, status FROM job_cards WHERE id = $1',
    [jobCardId]
  );

  if (jobCheck.rows.length === 0) {
    const error = new Error('Job card not found');
    error.statusCode = 404;
    throw error;
  }

  const jobStatus = jobCheck.rows[0].status;

  // VALIDATION: Only pending jobs can be deleted
  if (jobStatus !== 'pending') {
    const error = new Error(
      `Cannot delete job card with status '${jobStatus}'. Only pending jobs can be deleted.`
    );
    error.statusCode = 403; // Forbidden
    throw error;
  }

  // DELETE JOB CARD
  // CASCADE on job_status_history will auto-delete history records
  await pool.query('DELETE FROM job_cards WHERE id = $1', [jobCardId]);

  return {
    message: 'Job card deleted successfully',
    deletedId: jobCardId
  };
};

```

Delete Restriction:

- **Only pending jobs:** Once work starts, job becomes part of historical record
- **Cascade delete:** `job_status_history` records automatically deleted
- **Supervisor only:** Route-level authorization enforces this

6.3 Statistics Service

javascript

```
const getJobCardStatistics = async () => {
  // Query 1: Overall stats
  const overallStatsQuery = `
    SELECT
      COUNT(*) as total_jobs,
      COUNT(*) FILTER (WHERE status = 'pending') as pending_count,
      COUNT(*) FILTER (WHERE status = 'in_progress') as in_progress_count,
      COUNT(*) FILTER (WHERE status = 'completed') as completed_count
    FROM job_cards
  `;

  // Query 2: Priority breakdown (active jobs only)
  const priorityStatsQuery = `
    SELECT
      COUNT(*) FILTER (WHERE priority = 'urgent') as urgent_count,
      COUNT(*) FILTER (WHERE priority = 'high') as high_count,
      COUNT(*) FILTER (WHERE priority = 'medium') as medium_count,
      COUNT(*) FILTER (WHERE priority = 'low') as low_count
    FROM job_cards
    WHERE status != 'completed'
  `;

  // Query 3: Today's scheduled jobs
  const todayJobsQuery = `
    SELECT COUNT(*) as today_scheduled
    FROM job_cards
    WHERE DATE(scheduled_date) = CURRENT_DATE
  `;

  // Query 4: Completed in last 7 days
  const weekCompletedQuery = `
    SELECT COUNT(*) as week_completed
    FROM job_cards
    WHERE status = 'completed'
    AND completed_at >= CURRENT_DATE - INTERVAL '7 days'
  `;

  // Query 5: Average completion time
  const avgCompletionTimeQuery = `
    SELECT
      ROUND(
        AVG(
          EXTRACT(EPOCH FROM (actual_end_time - actual_start_time)) / 3600
        )::numeric,
        2
      ) as avg_completion_hours
  `;
```

```
FROM job_cards
WHERE status = 'completed'
AND actual_start_time IS NOT NULL
AND actual_end_time IS NOT NULL
`;
```

// Query 6: Recent activity (last 10 status changes)

```
const recentActivityQuery = `
SELECT
  jsh.job_card_id,
  jc.title as job_title,
  jsh.status,
  jsh.changed_at,
  u.name as technician_name
FROM job_status_history jsh
JOIN job_cards jc ON jsh.job_card_id = jc.id
JOIN users u ON jsh.changed_by_user_id = u.id
ORDER BY jsh.changed_at DESC
LIMIT 10
`;
```

// Execute all queries in parallel for performance

```
const [
  overallStats,
  priorityStats,
  todayJobs,
  weekCompleted,
  avgCompletionTime,
  recentActivity
] = await Promise.all([
  pool.query(overallStatsQuery),
  pool.query(priorityStatsQuery),
  pool.query(todayJobsQuery),
  pool.query(weekCompletedQuery),
  pool.query(avgCompletionTimeQuery),
  pool.query(recentActivityQuery)
]);
```

// Format and return

```
return {
  total_jobs: parseInt(overallStats.rows[0].total_jobs),
  pending_jobs: parseInt(overallStats.rows[0].pending_count),
  in_progress_jobs: parseInt(overallStats.rows[0].in_progress_count),
  completed_jobs: parseInt(overallStats.rows[0].completed_count),

  urgent_count: parseInt(priorityStats.rows[0].urgent_count),
  high_count: parseInt(priorityStats.rows[0].high_count),
```

```
medium_count: parseInt(priorityStats.rows[0].medium_count),
low_count: parseInt(priorityStats.rows[0].low_count),

today_scheduled: parseInt(todayJobs.rows[0].today_scheduled),
week_completed: parseInt(weekCompleted.rows[0].week_completed),
avg_completion_hours: parseFloat(avgCompletionTime.rows[0].avg_completion_hours) || 0,

recent_activity: recentActivity.rows

};
};
```

Performance Optimization:

- **Promise.all():** Executes 6 queries in parallel (faster than sequential)
 - **Aggregate functions:** Database calculates counts (no app-level loops)
 - **Filtered aggregates:** `COUNT(*) FILTER (WHERE ...)` is PostgreSQL-specific (efficient)
-

7. Data Validation & Error Handling

7.1 Validation Strategy (Defense in Depth)

Validation occurs at **3 layers**:

1. DATABASE CONSTRAINTS

← Last line of defense

- NOT NULL

- CHECK (status IN (...))

- FOREIGN KEY constraints



2. SERVICE LAYER (Business Logic)

← Business rules

- Status transitions

- Foreign key existence checks

- Time logic (end > start)



3. MIDDLEWARE (Schema Validation)

← First barrier

- Joi schemas

- Data types, formats, lengths

- Required fields

7.2 Joi Schema Validation

javascript

```
// validators/jobCard.validator.js
const Joi = require('joi');

const createJobCardSchema = Joi.object({
  customer_id: Joi.number()
    .integer()
    .positive()
    .required()
    .messages({
      'number.base': 'customer_id must be a number',
      'number.positive': 'customer_id must be positive',
      'any.required': 'customer_id is required'
    }),

  technician_id: Joi.number()
    .integer()
    .positive()
    .required()
    .messages({
      'number.base': 'technician_id must be a number',
      'number.positive': 'technician_id must be positive',
      'any.required': 'technician_id is required'
    }),

  title: Joi.string()
    .min(3)
    .max(200)
    .required()
    .messages({
      'string.empty': 'title is required',
      'string.min': 'title must be at least 3 characters',
      'string.max': 'title cannot exceed 200 characters'
    }),

  description: Joi.string()
    .max(2000)
    .allow("", null),

  priority: Joi.string()
    .valid('low', 'medium', 'high', 'urgent')
    .default('medium')
    .messages({
      'any.only': 'priority must be one of: low, medium, high, urgent'
    }),

  scheduled_date: Joi.string()
```

```
.isoDate()
.required()
.messages({
  'string.isoDate': 'scheduled_date must be a valid ISO 8601 date',
  'any.required': 'scheduled_date is required'
}),

estimated_duration: Joi.number()
  .integer()
  .positive()
  .max(1440) // Max 24 hours in minutes
  .allow(null)
  .messages({
    'number.max': 'estimated_duration cannot exceed 1440 minutes (24 hours)'
  }),

notes: Joi.string()
  .max(1000)
  .allow("", null)
});
```

Validation Middleware:

javascript

```
// middleware/validateRequest.js
const validateRequest = (schema) => {
  return (req, res, next) => {
    const { error, value } = schema.validate(req.body, {
      abortEarly: false, // Return ALL errors, not just first
      stripUnknown: true // Remove fields not in schema
    });

    if (error) {
      const errors = error.details.map(detail => ({
        field: detail.path.join('.'),
        message: detail.message
      }));

      return res.status(400).json({
        success: false,
        error: 'Validation failed',
        details: errors
      });
    }

    // Replace req.body with validated and sanitized value
    req.body = value;
    next();
  };
};
```

Example Error Response:

```
json
{
  "success": false,
  "error": "Validation failed",
  "details": [
    {
      "field": "title",
      "message": "title must be at least 3 characters"
    },
    {
      "field": "scheduled_date",
      "message": "scheduled_date is required"
    }
  ]
}
```

7.3 Error Handling Strategy

```
javascript

// Global error handler (last middleware in server.js)
app.use((err, req, res, next) => {
  console.error('Unhandled error:', err);

  res.status(err.statusCode || 500).json({
    success: false,
    error: err.message || 'Internal server error',
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
  });
});
```

Error Propagation:

1. **Service layer** throws errors with `statusCode`
2. **Controller** catches and forwards to error handler
3. **Error handler** sends formatted response

Example:

```
javascript

// Service
const error = new Error('Job card not found');
error.statusCode = 404;
throw error;

// Controller
try {
  const job = await jobCardService.getById(id);
  res.json({ success: true, data: { jobCard: job } });
} catch (error) {
  const statusCode = error.statusCode || 500;
  res.status(statusCode).json({
    success: false,
    error: error.message
  });
}
```

7.4 HTTP Status Codes Used

Code	Meaning	When Used
200	OK	Successful GET, PATCH, DELETE
201	Created	Successful POST (new resource created)
400	Bad Request	Validation errors, business rule violations
401	Unauthorized	Missing/invalid token
403	Forbidden	Insufficient permissions, completed job modification
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate email, invalid status transition
500	Server Error	Database errors, unexpected exceptions

8. Security Implementation

8.1 SQL Injection Prevention

All queries use parameterized statements:

```
javascript

// ❌ VULNERABLE (string concatenation)
const query = `SELECT * FROM users WHERE email = '${email}'`;

// ✅ SAFE (parameterized query)
const query = 'SELECT * FROM users WHERE email = $1';
const result = await pool.query(query, [email]);
```

pg library automatically escapes parameters, preventing SQL injection.

8.2 Password Security

```
javascript

// Registration
const passwordHash = await bcrypt.hash(password, 10);

// Login
const isValid = await bcrypt.compare(password, storedHash);
```

Bcrypt properties:

- **Salt:** Random value added to each password (prevents rainbow tables)
- **Slow:** Intentionally slow hashing (mitigates brute-force)
- **Adaptive:** Can increase rounds as hardware improves

8.3 JWT Security

```
javascript
// Token contains NO sensitive data (just identifiers)
{
  userId: 123,
  email: "tech@copycat.com",
  role: "technician",
  iat: 1706184000,
  exp: 1706270400
}
```

Security measures:

1. **Secret in environment variable:** Never committed to Git
2. **24h expiry:** Limits damage if token stolen
3. **No password in payload:** Token is base64 (not encrypted)
4. **HTTPS only** (in production): Prevents token interception

8.4 CORS Configuration

```
javascript
// server.js
const cors = require('cors');

app.use(cors({
  origin: process.env.CORS_ORIGIN || 'http://localhost:5173',
  credentials: true
}));
```

Production config:

- **Whitelist frontend domain:** Only allow requests from known origin
- **credentials: true:** Allow cookies (if needed for sessions)

8.5 Rate Limiting (Future Enhancement)

```
javascript

// Would add:
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100 // 100 requests per window
});

app.use('/api/v1/auth', limiter);
```

8.6 Input Sanitization

Joi validation strips unknown fields:

```
javascript

schema.validate(req.body, {
  stripUnknown: true // Remove fields not in schema
});
```

Prevents mass-assignment vulnerabilities:

```
javascript

// Client sends:
{
  title: "Hack",
  role: "supervisor" // Trying to escalate privileges
}

// Joi strips 'role' (not in schema)
// Only 'title' reaches database
```

9. Key Technical Decisions

9.1 Why PostgreSQL over MongoDB?

Requirement	PostgreSQL	MongoDB
Relationships	Strong (foreign keys, joins)	Weak (manual references)
Data Integrity	ACID transactions	Eventually consistent

Requirement	PostgreSQL	MongoDB
Schema Validation	Built-in constraints	Optional validation
Complex Queries	SQL (very expressive)	Aggregation pipeline (verbose)

Decision: PostgreSQL because:

- Job cards have **strong relationships** (customer, technician)
- Need **ACID guarantees** (audit trail accuracy)
- **Check constraints** enforce business rules at DB level

9.2 Why JWT over Sessions?

Aspect	JWT	Sessions
Server State	Stateless	Stateful (in-memory or Redis)
Scalability	Easy (no shared state)	Hard (session replication needed)
Performance	No DB lookup per request	DB/cache lookup per request
Mobile-Friendly	Yes (token in headers)	Harder (cookie management)

Decision: JWT because:

- **Horizontal scaling:** No session store bottleneck
- **Microservices-ready:** Token can be verified by any service
- **Mobile app future:** Easier to adapt

9.3 Why Layered Architecture?

Alternatives considered:

1. **MVC:** Controllers directly query database
2. **Clean Architecture:** Complex (overkill for this size)

Why Layered:

- **Separation of concerns:** HTTP logic ≠ business logic
- **Testable:** Can mock database for service tests
- **Maintainable:** Changes in one layer don't cascade

9.4 Why Dynamic Query Building?

Alternative: Fixed queries for each filter combination

Problem: Exponential combinations (status × technician × date × priority...)

Solution: Build queries dynamically based on provided filters

Trade-off:

-  **Pro:** Flexible, fewer queries to maintain
-  **Con:** Slightly more complex code
- **Verdict:** Worth it for API flexibility

9.5 Why Separate Complete Endpoint?

Alternative: PATCH `/job-cards/:id` with `{ status: 'completed', ... }`

Why Separate:

1. **Semantic clarity:** `/complete` is self-documenting
2. **Different validation:** Requires `work_performed`, optional signature
3. **Business logic:** Completion has unique requirements
4. **API discoverability:** Clients know there's a special completion flow

9.6 Why Immutable Completed Jobs?

Alternative: Allow updates to completed jobs

Problems:

1. **Audit trail tampering:** Could change historical records
2. **Reporting inaccuracy:** Statistics would be unreliable
3. **Legal issues:** Completed work is a legal record

Solution: Make completed jobs read-only (enforced in service layer)

10. Code Walkthrough

10.1 Request Lifecycle - Complete Example

Let's trace a **technician completing a job**:

1. Client Request

```
javascript
```

POST /api/v1/job-cards/42/complete

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

```
{
  "work_performed": "Replaced toner cartridge, cleaned print heads, tested 100 pages",
  "actual_end_time": "2026-02-16T14:30:00Z",
  "notes": "Customer reported slow printing - resolved by firmware update"
}
```

2. Middleware Pipeline

javascript

```
// authenticate.js
const token = req.headers.authorization.substring(7);
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = { userId: 5, email: "john@copycat.com", role: "technician" };

// jobCardAuth.js (verifyJobCardOwnership)
const jobCheck = await pool.query(
  'SELECT technician_id FROM job_cards WHERE id = $1',
  [42]
);
// jobCheck.rows[0].technician_id === 5 ✓ (matches req.user.userId)

// validateRequest.js
const { error, value } = completeJobCardSchema.validate(req.body);
// ✓ work_performed present, correct format
req.body = value; // Sanitized data
```

3. Controller

javascript

```
// controllers/jobCard.controller.js
const completeJobCard = async (req, res) => {
  try {
    const jobCardId = parseInt(req.params.id); // 42
    const completionData = req.body;

    const completedJobCard = await jobCardService.completeJobCard(
      jobCardId,
      completionData
    );

    res.json({
      success: true,
      message: 'Job card completed successfully',
      data: { jobCard: completedJobCard }
    });
  } catch (error) {
    // ...error handling
  }
};
```

4. Service Layer

javascript

```

// services/jobCard.service.js
const completeJobCard = async (jobCardId, completionData) => {
  // Fetch current status
  const currentJob = await pool.query(
    'SELECT status, actual_start_time FROM job_cards WHERE id = $1',
    [42]
  );

  // status: 'in_progress', actual_start_time: '2026-02-16T08:00:00Z'

  // Validate: not already completed ✓
  // Validate: not pending ✓
  // Validate: work_performed present ✓ (>= 10 chars)

  // Update to completed
  const query = `
    UPDATE job_cards
    SET
      status = 'completed',
      actual_end_time = $1,
      work_performed = $2,
      notes = $3,
      completed_at = CURRENT_TIMESTAMP
    WHERE id = $4
  `;

  await pool.query(query, [
    '2026-02-16T14:30:00Z',
    'Replaced toner cartridge...',
    'Customer reported slow printing...',
    42
  ]);

  // Trigger fires: INSERT INTO job_status_history (...)

  // Fetch complete job with JOINS
  return await getJobCardById(42);
};

```

5. Database Operations

sql

-- 1. Service queries current status

```
SELECT status, actual_start_time FROM job_cards WHERE id = 42;
```

-- 2. Service updates to completed

```
UPDATE job_cards SET
  status = 'completed',
  actual_end_time = '2026-02-16T14:30:00Z',
  work_performed = 'Replaced toner cartridge...',
  notes = 'Customer reported slow printing...',
  completed_at = CURRENT_TIMESTAMP
WHERE id = 42;
```

-- 3. Trigger fires automatically

```
INSERT INTO job_status_history (job_card_id, status, changed_by_user_id)
VALUES (42, 'completed', 5);
```

-- 4. Service fetches complete result

```
SELECT
  jc.*,
  c.name as customer_name, c.address, c.phone, c.email,
  u.name as technician_name, u.email as technician_email
FROM job_cards jc
JOIN customers c ON jc.customer_id = c.id
JOIN users u ON jc.technician_id = u.id
WHERE jc.id = 42;
```

6. Response to Client

json

```
{
  "success": true,
  "message": "Job card completed successfully",
  "data": {
    "jobCard": {
      "id": 42,
      "title": "Photocopier Maintenance",
      "status": "completed",
      "priority": "medium",
      "scheduled_date": "2026-02-16T08:00:00Z",
      "actual_start_time": "2026-02-16T08:00:00Z",
      "actual_end_time": "2026-02-16T14:30:00Z",
      "work_performed": "Replaced toner cartridge...",
      "notes": "Customer reported slow printing...",
      "completed_at": "2026-02-16T14:30:15Z",
      "customer": {
        "id": 7,
        "name": "Acme Corp",
        "address": "123 Business St, Nairobi",
        "phone": "+254712345678"
      },
      "technician": {
        "id": 5,
        "name": "John Doe",
        "email": "john@copycat.com"
      }
    }
  }
}
```

10.2 Key Files Explained

server.js (Application Entry Point)

javascript

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const pool = require('./src/config/database');

// Import routes
const authRoutes = require('./src/routes/auth.routes');
const userRoutes = require('./src/routes/user.routes');
const customerRoutes = require('./src/routes/customer.routes');
const jobCardRoutes = require('./src/routes/jobCard.routes');

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Routes
app.use('/api/v1/auth', authRoutes);
app.use('/api/v1/users', userRoutes);
app.use('/api/v1/customers', customerRoutes);
app.use('/api/v1/job-cards', jobCardRoutes);

// Health check
app.get('/api/v1/health', (req, res) => {
  res.json({ success: true, message: 'Server is running' });
});

// 404 handler
app.use((req, res) => {
  res.status(404).json({ success: false, error: 'Route not found' });
});

// Global error handler
app.use((err, req, res, next) => {
  console.error('Unhandled error:', err);
  res.status(err.statusCode || 500).json({
    success: false,
    error: err.message || 'Internal server error'
  });
});

// Start server
app.listen(PORT, () => {
```

```
console.log(`Server running on http://localhost:${PORT}`);
});
```

Key Points:

1. **Environment variables first:** `dotenv.config()` at top
2. **Middleware order matters:** CORS → body parsers → routes
3. **Error handlers last:** 404 → global error handler
4. **Health check:** Useful for monitoring/load balancers

config/database.js (Connection Pool)

```
javascript

const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});

// Test connection on startup
pool.query('SELECT NOW()', (err, res) => {
  if (err) {
    console.error('❌ Database connection failed:', err.message);
    process.exit(-1);
  } else {
    console.log('✅ Database connected successfully');
  }
});

// Handle unexpected errors
pool.on('error', (err) => {
  console.error('Unexpected database error:', err);
  process.exit(-1);
});

module.exports = pool;
```

Why Crash on Database Errors?

- **Fail-fast:** Better to crash than run with inconsistent state
- **Orchestration:** Process managers (PM2, Kubernetes) will auto-restart

- **Alerting:** Crashes trigger monitoring alerts
-

11. Testing & Deployment

11.1 Testing Strategy (Recommended)

Unit Tests (Services Layer)

```
javascript

// tests/unit/jobCard.service.test.js
describe('JobCardService', () => {
  describe('completeJobCard', () => {
    it('should throw error if job is pending', async () => {
      // Mock database to return pending job
      // Call completeJobCard
      // Expect error: "Job must be started first"
    });

    it('should require work_performed field', async () => {
      // Mock in_progress job
      // Call completeJobCard without work_performed
      // Expect error: "work_performed is required"
    });

    it('should set completed_at timestamp', async () => {
      // Mock successful completion
      // Verify completed_at is set
    });
  });
});
```

Integration Tests (API Endpoints)

```
javascript
```

```
// tests/integration/jobCard.routes.test.js
const request = require('supertest');
const app = require('../server');

describe('POST /api/v1/job-cards/:id/complete', () => {
  it('should return 401 without token', async () => {
    const response = await request(app)
      .post('/api/v1/job-cards/1/complete')
      .send({ work_performed: 'Test work' });

    expect(response.status).toBe(401);
  });

  it('should complete job with valid data', async () => {
    const token = 'valid-jwt-token';
    const response = await request(app)
      .post('/api/v1/job-cards/1/complete')
      .set('Authorization', `Bearer ${token}`)
      .send({
        work_performed: 'Replaced toner',
        actual_end_time: '2026-02-16T14:00:00Z'
      });

    expect(response.status).toBe(200);
    expect(response.body.success).toBe(true);
    expect(response.body.data.jobCard.status).toBe('completed');
  });
});
```

11.2 Environment Variables

```
bash
```

```
# .env (development)
NODE_ENV=development
PORT=5000
DATABASE_URL=postgresql://user:password@localhost:5432/jobcards_dev
JWT_SECRET=dev-secret-key-change-in-production
JWT_EXPIRY=24h
BCRYPT_ROUNDS=10
CORS_ORIGIN=http://localhost:5173

# .env.production
NODE_ENV=production
PORT=5000
DATABASE_URL=postgresql://user:password@prod-db-host:5432/jobcards_prod
JWT_SECRET=super-secret-production-key-256-bits
JWT_EXPIRY=24h
BCRYPT_ROUNDS=12
CORS_ORIGIN=https://jobcards.copycatgroup.com
```

11.3 Deployment Checklist

Pre-deployment:

- ☐ Environment variables configured
- ☐ Database migrations run
- ☐ JWT_SECRET is strong (256+ bits)
- ☐ CORS_ORIGIN set to production domain
- ☐ Database backups enabled
- ☐ HTTPS enforced
- ☐ Rate limiting enabled
- ☐ Logging configured (Winston/Bunyan)
- ☐ Health check endpoint working
- ☐ Error monitoring (Sentry/Bugsnag)

Deployment Platforms:

Platform	Pros	Cons
Railway	Easy setup, auto-deploy from Git	Limited free tier
Render	Free PostgreSQL database	Cold starts on free tier
Heroku	Battle-tested, good docs	Expensive for production
DigitalOcean	Full control, affordable	Manual setup required

11.4 Database Migration Strategy

Initial Migration:

```
sql

-- migrations/001_initial_schema.sql
CREATE TABLE users (...);
CREATE TABLE customers (...);
CREATE TABLE job_cards (...);
CREATE TABLE job_status_history (...);

-- Triggers
CREATE FUNCTION update_updated_at_column() ...
CREATE TRIGGER update_users_updated_at ...

CREATE FUNCTION log_job_status_change() ...
CREATE TRIGGER log_status_change ...
```

Future Migrations:

```
sql

-- migrations/002_add_customer_signature.sql
ALTER TABLE job_cards
ADD COLUMN customer_signature TEXT;

-- migrations/003_add_job_priority_index.sql
CREATE INDEX idx_job_cards_priority ON job_cards(priority);
```

Migration Tool: Use `node-pg-migrate` or `knex.js` for version control.

12. Future Improvements

12.1 Short-Term Enhancements

1. PDF Generation

```
javascript
```

```
// Generate job card as PDF
```

```
POST /api/v1/job-cards/:id/pdf
```

```
// Implementation:
```

```
const PDFDocument = require('pdfkit');
```

```
const generateJobCardPDF = async (jobCardId) => {
```

```
  const job = await getJobCardById(jobCardId);
```

```
  const doc = new PDFDocument();
```

```
  doc.fontSize(20).text('JOB CARD', { align: 'center' });
```

```
  doc.fontSize(12).text(`Job #${job.id}`);
```

```
  doc.text(`Customer: ${job.customer.name}`);
```

```
  doc.text(`Technician: ${job.technician.name}`);
```

```
  doc.text(`Work Performed: ${job.work_performed}`);
```

```
  if (job.customer_signature) {
```

```
    // Add signature image
```

```
    doc.image(Buffer.from(job.customer_signature, 'base64'));
```

```
  }
```

```
  doc.end();
```

```
  return doc;
```

```
};
```

2. Email Notifications

```
javascript
```

```
// Notify customer when job completed
```

```
const sendCompletionEmail = async (jobCardId) => {
```

```
  const job = await getJobCardById(jobCardId);
```

```
  await sendEmail({
```

```
    to: job.customer.email,
```

```
    subject: `Job Completed - ${job.title}`,
```

```
    body: `Dear ${job.customer.name}, your job has been completed...`
```

```
  });
```

```
};
```

3. File Uploads (Photos)

```
javascript
```

```
// Upload before/after photos
POST /api/v1/job-cards/:id/photos

// Use multer middleware + S3/Cloudinary storage
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });

router.post('/:id/photos', upload.array('photos', 5), uploadPhotos);
```

12.2 Medium-Term Enhancements

1. Real-Time Updates (WebSockets)

```
javascript

// Notify supervisors when job status changes
const io = require('socket.io')(server);

io.on('connection', (socket) => {
  socket.on('subscribe-to-jobs', () => {
    socket.join('job-updates');
  });
});

// In service layer
const updateJobCard = async (jobCardId, updateData) => {
  // ... update logic

  if (updateData.status) {
    io.to('job-updates').emit('job-status-changed', {
      jobCardId,
      newStatus: updateData.status
    });
  }

  return updatedJob;
};
```

2. Advanced Filtering

```
javascript
```

```
// GET /api/v1/job-cards?
// status=pending,in_progress
// &priority=high,urgent
// &technician_id=5
// &scheduled_after=2026-02-01
// &scheduled_before=2026-02-28
// &customer_name=Acme
// &sort_by=priority
// &sort_order=desc
```

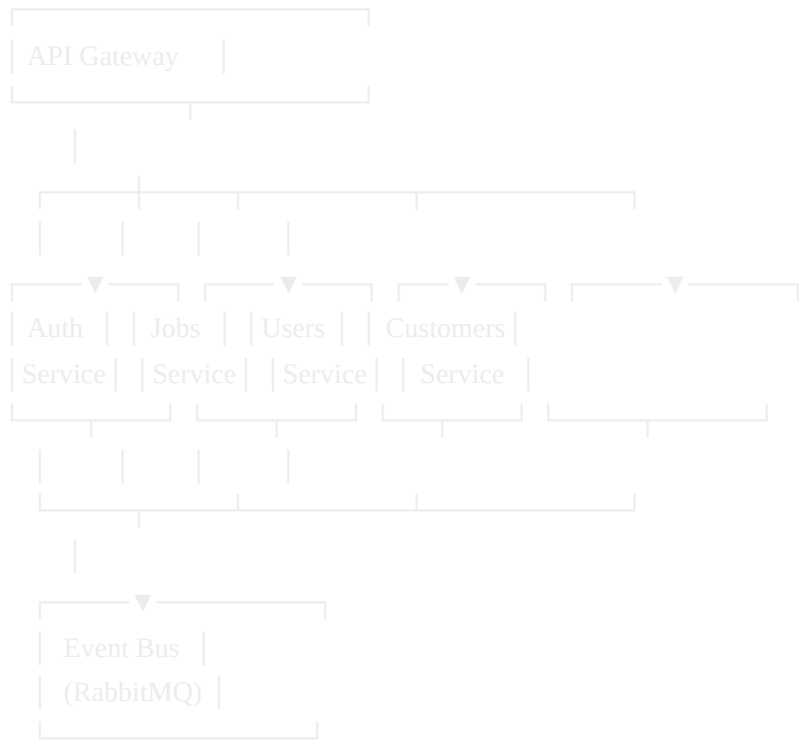
3. Batch Operations

```
javascript

// Assign multiple jobs to a technician
POST /api/v1/job-cards/batch-assign
{
  "job_ids": [1, 2, 3, 4, 5],
  "technician_id": 7
}
```

12.3 Long-Term Enhancements

1. Microservices Architecture



2. Caching Layer

```
javascript
```

```
// Cache frequently accessed data
const redis = require('redis');
const client = redis.createClient();

const getJobCardById = async (jobCardId) => {
  // Check cache first
  const cached = await client.get(`job:${jobCardId}`);
  if (cached) return JSON.parse(cached);

  // Fetch from database
  const job = await pool.query('SELECT ...');

  // Cache for 5 minutes
  await client.setex(`job:${jobCardId}`, 300, JSON.stringify(job));

  return job;
};
```

3. GraphQL API

```
graphql

query GetJobCard($id: ID!) {
  jobCard(id: $id) {
    id
    title
    status
    customer {
      name
      phone
    }
    technician {
      name
      email
    }
  }
}
```

4. Mobile App Backend

```
javascript
```

```
// Push notifications
const admin = require('firebase-admin');

const notifyTechnicianOfNewJob = async (technicianId, jobId) => {
  const fcmToken = await getUserFCMToken(technicianId);

  await admin.messaging().send({
    token: fcmToken,
    notification: {
      title: 'New Job Assigned',
      body: 'You have a new job card to complete'
    },
    data: { jobCardId: jobId.toString() }
  });
};
```

12.4 Performance Optimizations

1. Database Indexing

```
sql

-- Composite indexes for common queries
CREATE INDEX idx_job_cards_status_technician
  ON job_cards(status, technician_id);

CREATE INDEX idx_job_cards_scheduled_status
  ON job_cards(scheduled_date, status);
```

2. Query Optimization

```
javascript

// Before: N+1 query problem
const jobs = await getAllJobCards(); // 1 query
for (let job of jobs) {
  job.customer = await getCustomer(job.customer_id); // N queries
}

// After: Single query with JOIN
const jobs = await pool.query(`
  SELECT jc.*, c.name, c.phone, u.name as tech_name
  FROM job_cards jc
  JOIN customers c ON jc.customer_id = c.id
  JOIN users u ON jc.technician_id = u.id
`);
```

3. Pagination Best Practices

```
javascript

// Cursor-based pagination (better for large datasets)
GET /api/v1/job-cards?cursor=42&limit=20

// Returns jobs with id > 42
SELECT * FROM job_cards WHERE id > $1 LIMIT $2;
```

13. Conclusion

13.1 System Highlights

Architecture:

- ✓ Layered architecture (Controllers → Services → Database)
- ✓ RESTful API design with consistent responses
- ✓ Role-based access control (Supervisor vs Technician)
- ✓ Middleware pipeline for authentication, authorization, validation

Database:

- ✓ Relational model with foreign keys and constraints
- ✓ Automatic audit trail (triggers for status history)
- ✓ Immutable completed jobs (data integrity)
- ✓ Connection pooling for performance

Security:

- ✓ JWT-based stateless authentication
- ✓ Bcrypt password hashing
- ✓ SQL injection prevention (parameterized queries)
- ✓ Input validation at multiple layers

Business Logic:

- ✓ State machine for job status transitions
- ✓ Time validation (end time > start time)
- ✓ Ownership verification (technicians only access their jobs)
- ✓ Delete restrictions (only pending jobs)

13.2 What Makes This System Production-Ready?

1. **Error Handling:** Custom errors with status codes
2. **Validation:** Joi schemas + service-layer business rules + database constraints
3. **Logging:** Console logging (production would use Winston)
4. **Testing:** Designed for testability (service layer isolated)
5. **Documentation:** API endpoints self-documenting through routes
6. **Scalability:** Stateless authentication, connection pooling
7. **Maintainability:** Separation of concerns, consistent patterns

13.3 Lessons Learned

What Went Well:

- Middleware pipeline makes code very readable
- Database triggers automate audit trail (one less thing to forget)
- Service layer keeps business logic centralized

Challenges:

- Dynamic query building is complex (but necessary)
- JWT expiry handling requires frontend coordination
- Testing requires mocking database (not implemented yet)

What I'd Do Differently:

- Add caching for statistics queries (they're slow)
 - Implement soft deletes instead of hard deletes
 - Add more granular logging (request IDs, timing)
-

14. Presentation Tips

14.1 Demo Flow Recommendation

1. Start with Architecture Diagram (5 min)

- Show layered architecture
- Explain request flow
- Highlight separation of concerns

2. Live Database Demo (10 min)

- Show tables in pgAdmin
- Run sample queries
- Demonstrate triggers firing

3. API Demo with Postman (20 min)

- Login → get token
- Create job card (supervisor)
- Start job (technician)
- Complete job (technician)
- Try invalid operations (show error handling)

4. Code Walkthrough (30 min)

- Show middleware chain
- Explain service layer logic
- Point out validation layers
- Highlight security measures

5. Q&A Preparation (rest of time)

14.2 Common Questions & Answers

Q: Why not use an ORM like Sequelize?

A: I chose raw SQL for:

- **Performance:** No abstraction overhead
- **Control:** Complex queries are easier to optimize
- **Learning:** Better understanding of SQL
- **Trade-off:** More boilerplate, but more explicit

Q: How do you handle database migrations?

A: Currently manual SQL scripts. In production, I'd use `node-pg-migrate` for versioned migrations with rollback support.

Q: What about horizontal scaling?

A: JWT is stateless, so multiple backend instances can run behind a load balancer. Database connection pooling prevents bottlenecks.

Q: How do you prevent race conditions?

A: PostgreSQL's ACID transactions handle this. For example, updating job status is atomic. For critical operations, I'd add row-level locking:

```
sql
```

```
SELECT * FROM job_cards WHERE id = $1 FOR UPDATE;
```

Q: Why not use TypeScript?

A: Time constraint for this project. TypeScript would add type safety, better IDE support, and catch bugs earlier. Definitely recommended for production.

Q: How do you handle file uploads?

A: Not implemented yet. Would use `multer` for multipart/form-data, store files in S3/Cloudinary, save URLs in database.

Q: What about logging and monitoring?

A: Currently console.log. Production would use:

- **Winston/Bunyan** for structured logging
 - **Sentry** for error tracking
 - **Prometheus** for metrics
 - **Grafana** for dashboards
-

15. Appendix

15.1 Complete API Reference

See separate API documentation file (generated from Postman collection).

15.2 Database Schema SQL

See `migrations/` directory for complete schema.

15.3 Environment Setup Guide

```
bash
```

1. Clone repository

```
git clone <repo-url>
```

```
cd backend
```

2. Install dependencies

```
npm install
```

3. Setup PostgreSQL

```
createdb jobcards_dev
```

4. Run migrations

```
psql jobcards_dev < migrations/001_initial_schema.sql
```

5. Create .env file

```
cp .env.example .env
```

Edit .env with your database credentials

6. Start server

```
npm run dev
```

15.4 Project Statistics

- **Lines of Code:** ~3,000 (backend only)
- **API Endpoints:** 24
- **Database Tables:** 4
- **Middleware Functions:** 5
- **Service Functions:** 20+
- **Validators:** 6 schemas

END OF BACKEND DOCUMENTATION

This document serves as comprehensive technical documentation for the Job Card Management System backend. It covers architecture, design decisions, code walkthroughs, and deployment considerations suitable for presentation to senior software engineers.